ENTREGA FINAL DO MVP · 3ª E ÚLTIMA RODADA

MyGlico: Algoritmos e Técnicas de Ciência de Dados

Da estatística descritiva à detecção de anomalias e à clusterização de perfis glicêmicos

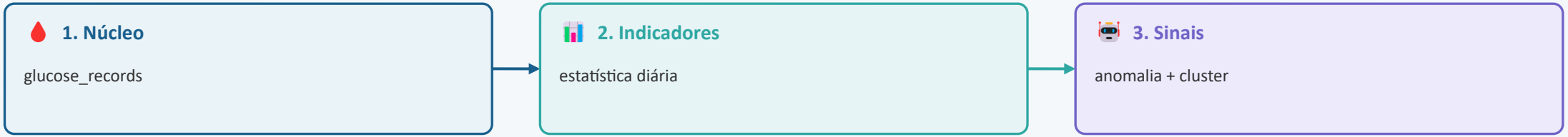
Estatística descritiva | Detecção de anomalias | Clusterização | Reprodutibilidade

🎯 Foco desta entrega

Explicar como o MyGlico lê glucose_records normalizados e produz indicadores clínicos, sinais de anomalia e perfis de dia, sempre com cada resultado rastreável à sua origem.

🏁 O que esta entrega fecha

Esta é a entrega final do MVP: fecha o ciclo problema, dados, arquitetura e algoritmos, com a camada analítica já funcionando sobre a base construída.



Felipe Rodrigues Calé
UFRJ • Instituto de Computação

RECAPITULAÇÃO

Da base confiável ao indicador clínico

As rodadas anteriores entregaram o dado confiável e rastreável; esta rodada final entrega o algoritmo que o transforma em leitura clínica

1. Núcleo clínico

glucose_records validado, rastreável e com origem conhecida

2. Classificação

cada medição vira hipo, pré-refeição, pós-refeição ou geral

3. Indicadores diários

média, desvio, % em cada faixa, por paciente e dia

4. Sinais avançados

IsolationForest + KMeans sobre os indicadores diários

Entregue nas rodadas anteriores

70 pacientes, 29.297 registros brutos e 13.531 medições normalizadas, com 196 apontamentos de qualidade e proveniência completa do dado bruto ao registro normalizado.

Entregue nesta rodada final

A camada analítica que fecha o MVP: estatística, detecção de anomalias e clusterização, com parâmetros e ambiente registrados.

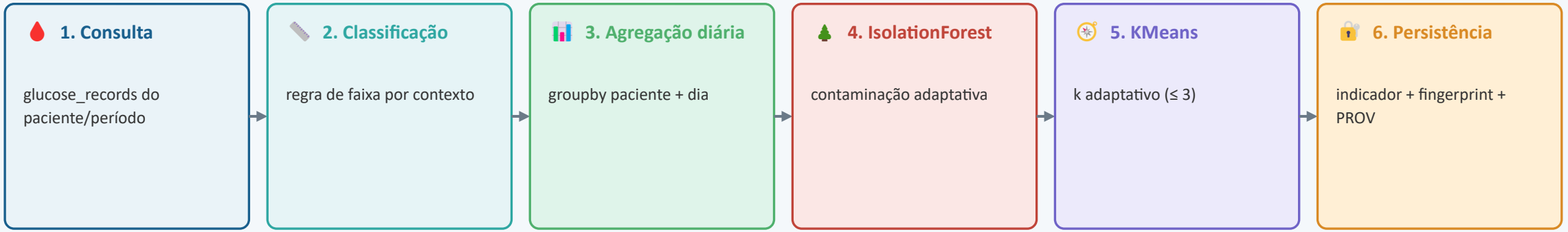
O que o MVP passa a fazer

Lê o dado clínico e produz indicadores, sinais de anomalia e perfis explicáveis, prontos para consulta no dashboard.

PIPELINE ANALÍTICO

Do registro normalizado à decisão assistida

Um único ponto de entrada (compute_analytics) encadeia cinco estágios determinísticos



Regra de execução: nada é sobrescrito sem registro, porque cada rodada gera um novo AnalysisRun com parâmetros, ambiente e hashes de entrada/saída.

Ponto de entrada único

compute_analytics(db, patient_id, start_date, end_date) roda para um paciente, um período, ou toda a base, sempre pelo mesmo caminho de código, registrado em app/services/analytics.py.

REGRAS CLÍNICAS

Classificação de faixas glicêmicas

A primeira camada não é estatística, e sim regra de domínio clínico aplicada linha a linha, com limiares alinhados à American Diabetes Association (ADA, 2026)

Hipoglicemia

< 70 mg/dL, em qualquer contexto

Pré-refeição

80–130 mg/dL

Pós-refeição

< 180 mg/dL

Geral (sem contexto)

70–180 mg/dL

Resultado: is_hypo, is_in_range e is_hyper são calculados linha a linha, antes de qualquer agregação. A estatística nunca decide o que é risco, apenas descreve o que a regra já classificou.

🍽 Contexto importa

O mesmo valor de glicose pode ser normal antes de uma refeição e alto depois dela, por isso o limiar superior depende do context_label do registro (pre_*, post_*, ou nenhum).

ESTATÍSTICA DESCRITIVA

Um indicador por paciente, por dia

Cada linha de DailyIndicator resume um dia inteiro de automonitoramento

Features calculadas

- Contagem de eventos no dia
- Média, mediana, mínimo e máximo
- Desvio-padrão e coeficiente de variação
- % do tempo em hipoglicemia
- % do tempo em faixa (geral, pré e pós-refeição)
- % do tempo em hiperglicemia
- Contagem de eventos baixos e altos

3.860
dias-paciente indicados

3,5
medições por dia, em média

161,5
mg/dL de glicemia média

30%
do tempo em faixa-alvo

Base real: esses números vêm da rodada de análise já registrada na base (analysis_run 8840c6f3), extraídos diretamente do dump atual, não de uma simulação.

IsolationForest: encontrando o dia fora do padrão

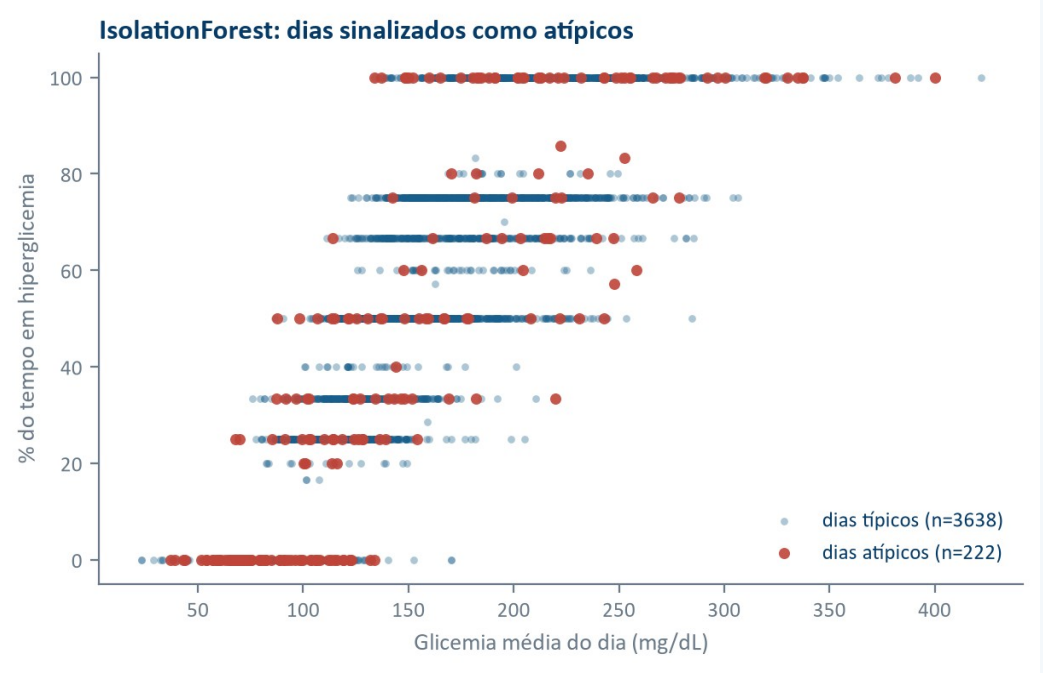
Um dia é atípico pela combinação dos indicadores, não por um limiar isolado

Como funciona

O algoritmo isola cada dia por cortes aleatórios nas features. Dias que se isolam com poucos cortes recebem um decision_function mais baixo e são marcados como anômalos.

Configuração

Features: avg_glucose, std_glucose, pct_hypo, pct_hyper, glucose_event_count.
contamination = min(0.2, max(0.05, 1/n))
random_state = 42
Só roda com ≥ 5 dias por paciente.



Por que essa técnica

A disciplina ensina IQR e Z-score (univariados) e cita IsolationForest/LOF como alternativas de ML (Módulo 2, Limpeza de Dados). Aqui a anomalia é multivariada, ou seja, nasce da combinação de indicadores e não de uma variável isolada. Por isso foi selecionado o IsolationForest, que dispensa a noção de vizinhança do LOF e é mais robusto com poucos dias por paciente.

CLUSTERIZAÇÃO DE PERFIS

KMeans: agrupando dias por perfil glicêmico

Os perfis emergem dos mesmos indicadores usados na detecção de anomalias

Onde: **app/services/analytics.py** (KMeans) → **app/services/reports.py** (_classify_cluster_profile) → **PatientDetailPage.jsx** ("Perfil do dia")

Como funciona

KMeans particiona os dias de cada paciente em até k=3 grupos brutos. Uma regra separada (_classify_cluster_profile) traduz a média de cada grupo em 1 de 5 perfis clínicos possíveis, no máximo 3 por paciente.

Configuração

Mesmas features da detecção de anomalias.

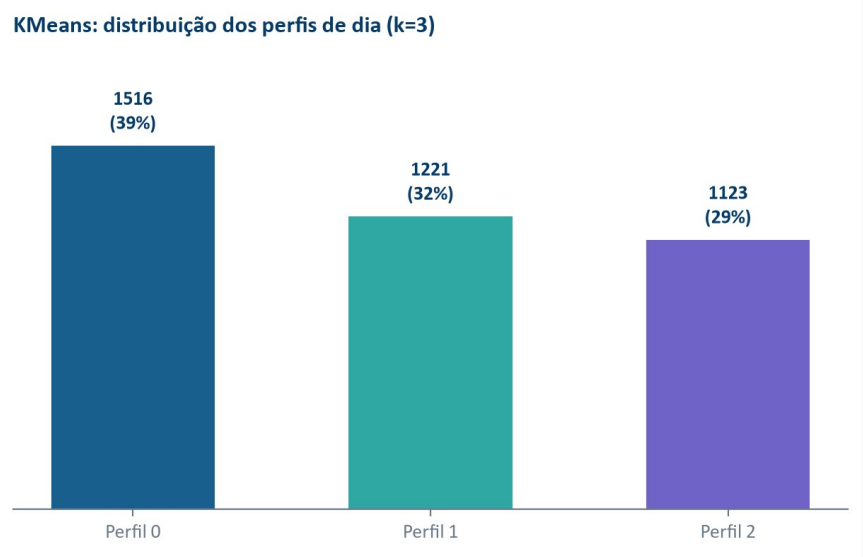
k = min(3, nº de dias do paciente)

n_init = 10 · random_state = 42

Só roda com ≥ 3 dias por paciente.

Perfis do dia: 5 possíveis, até 3 por paciente

Estável · Hiper predominante · Hipo frequente · Variável · Intermediário



Por que essa técnica

As features são contínuas e de baixa dimensão, e o objetivo é ter poucos perfis fixos e interpretáveis por paciente (k ≤ 3): o cenário clássico para clusterização por centróides. O KMeans é simples, determinístico com random_state fixo e cada centróide já resume o perfil médio do grupo, o que mantém o resultado explicável. A escolha se apoia no método do cotovelo e no silhouette score, e conversa com a clusterização vista na disciplina (Módulo 4).

REPRODUTIBILIDADE

Todo resultado sabe como foi produzido

Cada AnalysisRun é uma cápsula de parâmetros, ambiente e hashes, não apenas um resultado

🔢 Fingerprint de entrada e saída

input_fingerprint e output_fingerprint são SHA-256 sobre os registros usados e os indicadores gerados, então qualquer mudança no dado ou no resultado muda o hash.

📜 Parâmetros registrados

limiares clínicos, configuração do IsolationForest e do KMeans, tudo em parameters_json.

💻 Ambiente capturado

versão do Python, do SO, do banco e das bibliotecas (numpy, pandas, scikit-learn, SQLAlchemy) em environment_json.

🚀 Versão do algoritmo e do código

algorithm_name/version ("daily_glycemic_summary", v2) e o commit git da execução.

Resultado: duas rodadas podem ser comparadas lado a lado, e é possível saber exatamente o que mudou entre elas: o dado, o parâmetro ou o código.

📎 **app/services/analytics.py**

A proveniência da importação, em PROV-O

dataset-import e dataset-normalization aparecem como atividades PROV, ligadas ao mesmo lote de importação

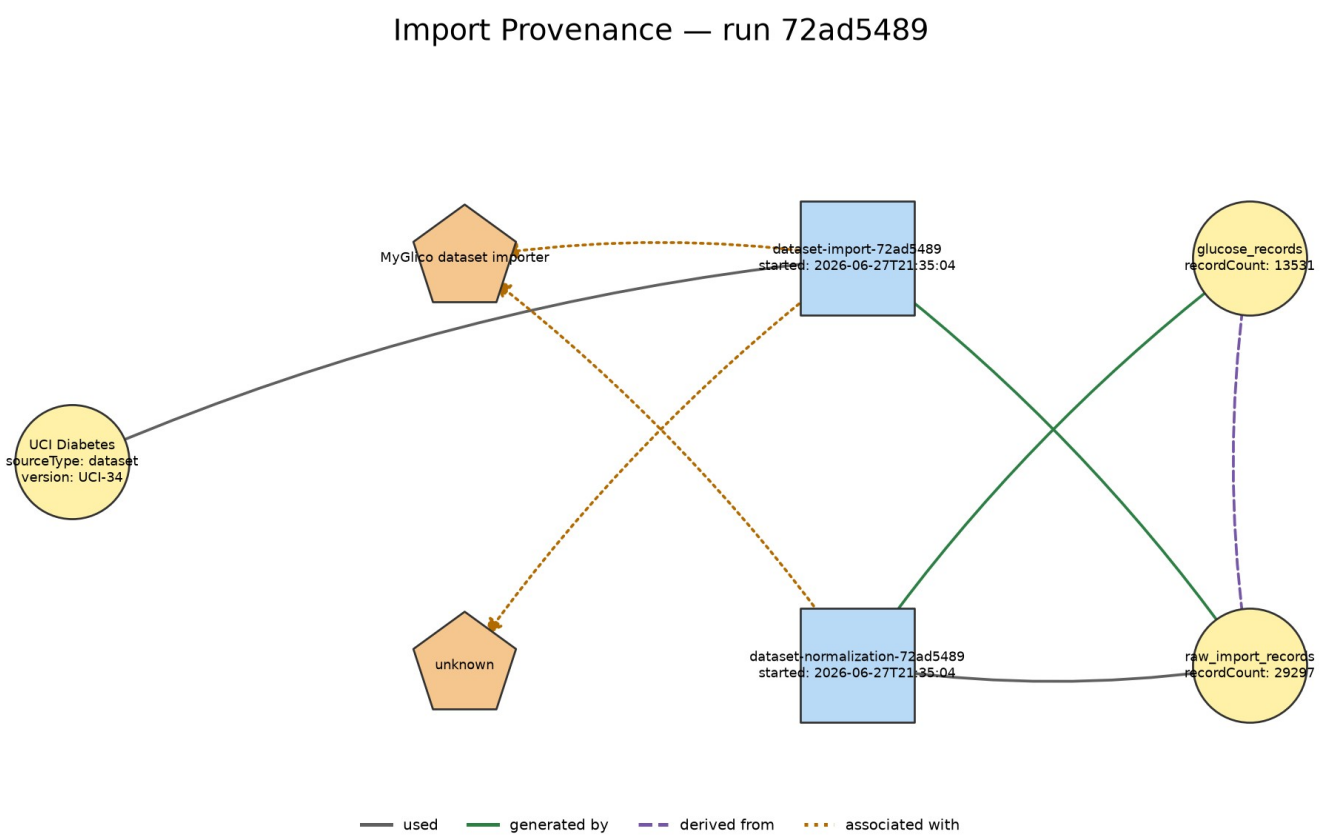


Diagrama gerado a partir do artefato real artifacts/provenance/import-72ad5489-….ttl deste lote, via scripts/render_provenance_png.py.

A proveniência do algoritmo, em PROV-O

compute_daily_indicators, detect_anomalies e cluster_days aparecem como atividades PROV, ligadas ao mesmo run

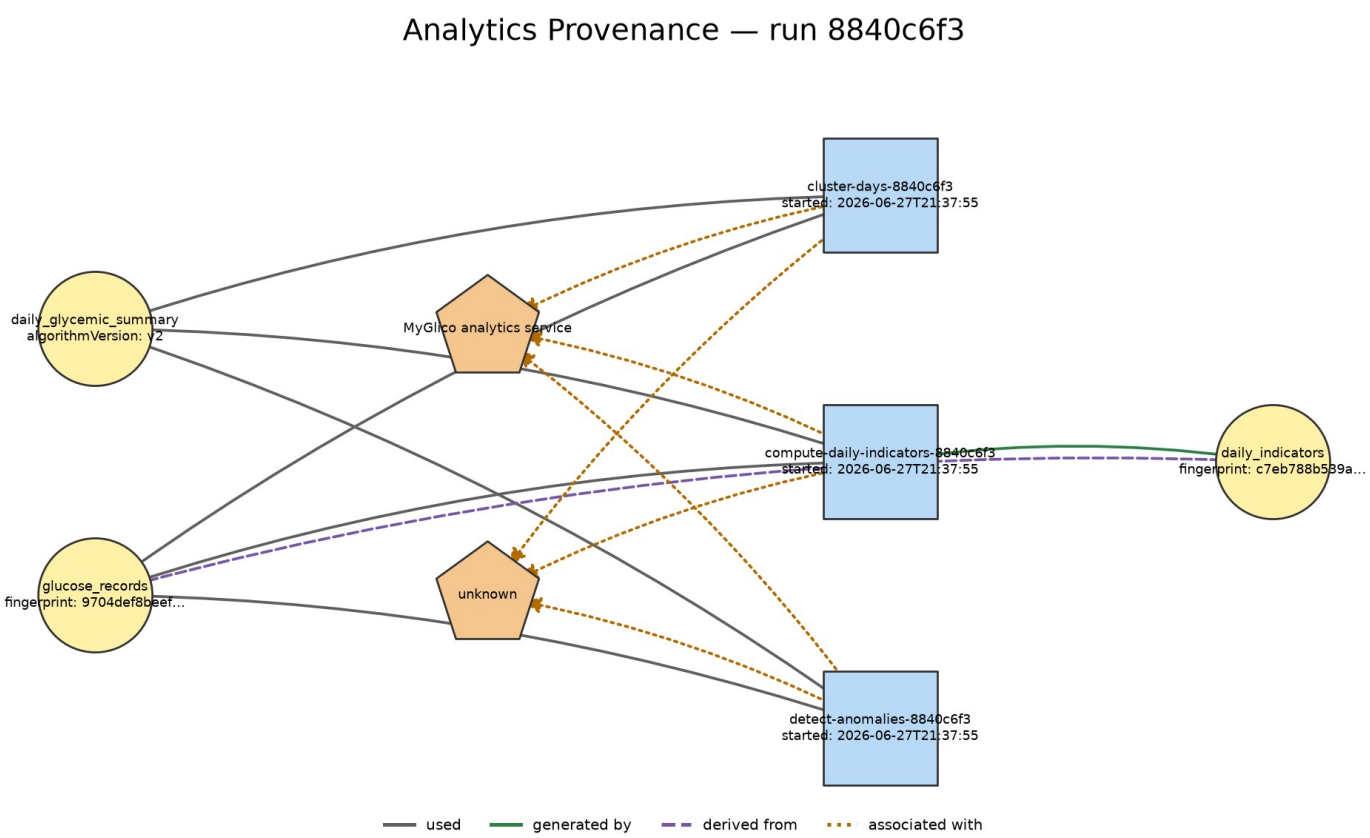


Diagrama gerado a partir do artefato real artifacts/provenance/analytics-8840c6f3-….ttl desta rodada, via scripts/render_provenance_png.py.

CAMADA SEMÂNTICA

Do core relacional ao RDF, com ontologia própria

Um mapeamento declarativo traduz o dado clínico em grafo, sem tocar na API em produção

🗺️ Mapeamento R2RML

Cada tabela do core relacional (patients, glucose_records, raw_import_records, data_sources) é mapeada para RDF pelo vocabulário mygv:, sem duplicar lógica de negócio.

🧬 Ontologia própria (mygv)

Estende a Diabetes Pharmacology Ontology (DPO) e o PROV-O, definindo classes como Patient, GlucoseMeasurement e DailyIndicator, e propriedades como pctHypo e inputFingerprint.

🔌 Motor externo (R2RML)

O RDF é gerado por um motor R2RML externo (ex.: Ontop), não pela API viva, mantendo a API leve e o grafo consultável por qualquer ferramenta SPARQL.

🔗 Interoperabilidade

O mesmo grafo pode ser consultado, combinado e publicado com padrões abertos (RDF, OWL, SPARQL), sem depender do frontend do MyGlico.

Resultado: o core clínico e sua proveniência viram um único grafo RDF navegável e citável, sem custo de manutenção na API em produção.

📎 **mappings/myglico-core-r2rml-juma.ttl · ontology/mygv-dpo-extension.owl.ttl**

LIMITAÇÕES E PRÓXIMOS PASSOS

O que o algoritmo ainda não faz

Ser honesto sobre o escopo atual orienta o que vem a seguir

Sem validação clínica

Os clusters e anomalias não foram revisados por um profissional de saúde, então são candidatos, não diagnósticos.

⚠ Regras de perfil não validadas

O cluster_label bruto não tem nome, mas o MyGlico já traduz cada cluster em um perfil (estável, hiper predominante, hipo frequente, variável, intermediário) por regras sobre a média do cluster. Os limiares dessas regras (ex.: hiper ≥ 60%) foram definidos por nós, não por um profissional de saúde.

⚠ Sem teste de contraste

Não há, ainda, um conjunto de dias sintéticos conhecidos para validar a sensibilidade do detector limiar por limiar.

🚀 Próximo passo

Expor anomaly_flag e cluster_label no frontend, e desenhar um contraste mínimo para validar o IsolationForest.

Regra de evolução: nenhum sinal chega ao paciente sem, antes, ser auditável na proveniência.

FECHAMENTO · ENTREGA DO MVP

MVP entregue: base confiável e análise explicável

O MVP está completo: da ingestão do dado à análise, tudo registrado, rastreável e reproduzível

✅ O que o MVP entrega

Banco versionado e proveniência, estatística diária, IsolationForest para dias atípicos e KMeans para perfis, tudo com fingerprint e ambiente registrados.

🔗 Preparado para

Comparação entre rodadas, novos limiares clínicos, recomputação automática após entrada manual.

🚀 Trabalho futuro (pós-MVP)

Validação clínica dos perfis, exposição plena no frontend, teste de contraste para o detector de anomalias e suporte a registro glicêmico por entrada de dados não estruturados.

Mensagem final: Com este MVP, o MyGlico não apenas descreve o dado: ele aplica algoritmos sobre uma base confiável e explica, com evidência, como cada resultado nasceu.

Obrigado!

REFERÊNCIAS

Referências e base técnica

Fontes usadas para fundamentar os algoritmos e a leitura dos resultados

- LIU, F. T.; TING, K. M.; ZHOU, Z.-H. Isolation Forest. IEEE International Conference on Data Mining (ICDM), 2008.
- LLOYD, S. Least squares quantization in PCM. IEEE Transactions on Information Theory, 1982. Base do algoritmo k-means, implementado por scikit-learn.KMeans.
- PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 2011.
- KAHN, M. Diabetes [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5T59G.
- AMERICAN DIABETES ASSOCIATION PROFESSIONAL PRACTICE COMMITTEE FOR DIABETES et al. Glycemic Goals, Hypoglycemia, and Hyperglycemic Crises: Standards of Care in Diabetes, 2026. Diabetes Care, v. 49, Supplement 1, p. S132, S149, 2026. DOI: 10.2337/dc26-S006.
- CALÉ, Felipe Rodrigues. MYGLICO: plataforma inteligente para diário de controle glicêmico para pacientes com Diabetes Mellitus. Pré-projeto de mestrado, UFRJ, 2026.
- Base técnica local do projeto MyGlico: app/services/analytics.py, app/services/provenance.py, scripts/render_provenance_png.py.
- Camada semântica do MyGlico: mappings/myglico-core-r2rml-juma.ttl (mapeamento R2RML) e ontology/mygv-dpo-extension.owl.ttl (ontologia própria, estendendo DPO e PROV-O).
- SERRA, Sergio; ZAVALETA, Jorge. Fundamentos de Ciência de Dados. PPGI/UFRJ, material de disciplina (Módulo 2: Limpeza e Preparação de Dados; Módulo 4: Machine Learning).

Observação: os números apresentados vêm da rodada de análise já registrada na base (analysis_run 8840c6f3-fb98-44b1-95b1-ae4457b7f140), extraídos diretamente do dump atual do banco.